

Universidad de Costa Rica

Sede del Pacífico

Campus Arnoldo Ferreto Segura Pacífico

Curso: Algoritmos y Estructura de Datos

Proyecto:

Algoritmos recursivos

Juego de la Vida - Ecosistema costarricense

Integrantes:

David Mora Gómez - C5H441

Daniela Bustos Medina - C5D410

Jimena Conejo Artavia - C5E399

Profesor:

Marcos Venegas

Junio, 2026

Índice

Portada	1
Índice	2
1. Objetivos	3
2. Introducción	4
3. Desarrollo	5
3.1 Concepto de algoritmo y solución de problemas	5
3.2 Recursividad: definición y estructura	5
3.3 Caso base, caso recursivo y pila de llamadas	5
3.4 Recursividad, iteración y complejidad	5
3.5 Autómatas celulares	5
3.6 Juego de la Vida de Conway	5
3.7 Aplicación al ecosistema costarricense	5
3.8 Diseño del programa propuesto	5
3.9 Método recursivo para contar vecinos	5
3.10 Aporte propio del grupo	5
4. Conclusiones	16
5. Referencias bibliográficas	17

1. Objetivos

1.1 Objetivo general

Analizar los algoritmos recursivos y su aplicación en la simulación del Juego de la Vida, con el fin de fundamentar el desarrollo de un programa en Java que modele generaciones de microorganismos dentro de un ecosistema costarricense.

1.2 Objetivos específicos

- Explicar el concepto de recursividad, sus partes principales y su relación con la solución de problemas algorítmicos.
- Describir el funcionamiento del Juego de la Vida de Conway como autómata celular basado en reglas locales.
- Relacionar la simulación con un contexto costarricense mediante el uso de reservas, zonas protegidas y escenarios naturales del país.
- Identificar la importancia del caso base, el caso recursivo y el control de llamadas para evitar errores en programas recursivos.
- Proponer una base teórica clara para defender el método recursivo que cuenta los vecinos vivos de cada celda en el programa.

2. Introducción

Los algoritmos recursivos constituyen una de las herramientas mas importantes dentro del estudio de la programación y de las estructuras de datos, porque permiten expresar soluciones que se definen en términos de versiones mas pequenas del mismo problema. A diferencia de una solución puramente iterativa, la recursividad exige reconocer con precision cual es el caso base, cual es el caso recursivo y de que manera se reduce el problema en cada llamada. Esta forma de razonar aparece en busquedas, ordenamientos, arboles, recorridos, fractales, juegos logicos y simulaciones. Por esa razon, estudiar la recursividad no consiste solamente en aprender una técnica de código, sino en comprender una manera ordenada de dividir problemas y controlar su solución.

En este trabajo se analiza la recursividad tomando como aplicación el Juego de la Vida de Conway, un autómatas celular que simula el nacimiento, la sobrevivencia y la muerte de celulas en una matriz. Aunque sus reglas son sencillas, el comportamiento resultante puede ser complejo, ya que cada generación depende del estado de las celdas vecinas. Esta relación entre reglas locales y comportamiento global convierte al Juego de la Vida en un ejemplo apropiado para estudiar algoritmos, matrices, estados discretos y simulaciones. Además, el requisito técnico del proyecto solicita que el conteo de vecinos se realice mediante un método recursivo, por lo que el tema central queda integrado directamente con la implementación.

La investigacion se contextualiza en Costa Rica mediante una simulación de microorganismos en una reserva natural. Este enfoque permite que el programa no sea un ejemplo generico, sino una aplicación ambientada en áreas silvestres protegidas del país. Para elaborar el documento se revisaron libros de algoritmos, textos especializados en recursividad, publicaciones sobre autómatas celulares, el articulo clasico que presento el Juego de la Vida y fuentes institucionales relacionadas con la conservación costarricense. Con esto se busca cumplir el requisito académico de utilizar fuentes confiables, evitar paginas no reconocidas y redactar una explicación propia que pueda defenderse oralmente.

3. Desarrollo

3.1 Metodología de investigación

La metodología utilizada para esta investigación fue una revisión bibliográfica orientada al tema asignado: algoritmos recursivos. Primero se identificaron fuentes base sobre algoritmos y estructuras de datos, ya que la recursividad forma parte de los fundamentos de programación y análisis algorítmico. En este grupo se consultaron textos reconocidos como *Introduction to Algorithms* de Cormen, Leiserson, Rivest y Stein, el material de *Algorithms* de Sedgewick y Wayne, y el libro *Thinking Recursively with Java* de Roberts. Estas obras permiten explicar la recursividad desde el punto de vista teórico y también desde el punto de vista de la implementación.

En una segunda etapa se revisaron fuentes sobre autómatas celulares y el Juego de la Vida. El artículo de Martin Gardner en *Scientific American* se consideró una referencia histórica porque fue el medio que popularizó el juego propuesto por John Conway. También se utilizaron trabajos de Wolfram, Ilachinski, Adamatzky y Rendell para comprender por qué los autómatas celulares son importantes en simulación, sistemas complejos y teoría de la computación. Estas fuentes no se eligieron por cantidad, sino por pertinencia: permiten conectar las reglas simples del Juego de la Vida con conceptos más amplios como autoorganización, vecindad, evolución discreta y computación.

Finalmente se agregó una fuente institucional costarricense: el Sistema Nacional de Áreas de Conservación. Esta fuente se usó para justificar el contexto nacional del proyecto, específicamente la referencia a áreas silvestres protegidas. El criterio general fue evitar fuentes no académicas o no reconocidas. Por eso no se utilizó Wikipedia como referencia bibliográfica. La redacción del documento no copia definiciones extensas, sino que resume y explica los conceptos con palabras propias, relacionándolos con el programa que se desarrollará en Java.

3.2 Concepto de algoritmo y solución de problemas

Un algoritmo puede entenderse como una serie finita y ordenada de pasos que transforma datos de entrada en un resultado. En informática, esta idea es fundamental porque permite pasar de un problema expresado en lenguaje natural a una solución que puede ejecutarse por una computadora. Cormen et al. (2022) presentan los algoritmos como procedimientos rigurosos que pueden analizarse por su corrección, eficiencia y aplicación. Esto significa que no basta con que un programa funcione una vez; también debe ser comprensible, repetible y capaz de producir resultados correctos para los casos esperados.

En un curso de algoritmos y estructuras de datos, la solución de problemas requiere identificar qué información se recibe, qué salida se espera y qué operaciones intermedias son necesarias. Por ejemplo, en el Juego de la Vida, la entrada no es solamente el tamaño de la matriz. También incluye la ubicación inicial de los microorganismos, la cantidad de generaciones y el estado vivo o vacío de cada celda. La salida, por su parte, consiste en mostrar la evolución de la matriz y el resumen de nacimientos, sobrevivientes y muertes. De esta manera, el problema se vuelve concreto y puede dividirse en tareas pequeñas.

Sedgewick y Wayne (2011) resaltan que los algoritmos deben estudiarse tomando en cuenta su impacto práctico, no solo su forma matemática. Esta idea se ajusta al proyecto porque la simulación no se queda en la teoría: debe convertirse en un programa visible para el usuario. La

matriz, las reglas de vida y muerte, el conteo de vecinos y la interfaz gráfica son componentes que deben trabajar juntos. Por eso, aunque el tema principal sea la recursividad, el algoritmo completo también necesita organización de datos, validaciones y una presentación clara del resultado.

El valor de un algoritmo también está en que obliga a pensar antes de programar. Si el grupo comprende primero las reglas del problema, el código se vuelve más fácil de defender. En cambio, si se escribe el programa sin entender el proceso, cualquier cambio pequeño puede provocar errores. En este proyecto, el punto crítico es el conteo de vecinos de cada celda, porque de ese cálculo dependen todas las decisiones posteriores. La investigación sirve entonces como base para saber por qué la solución es correcta y como explicar cada parte en la defensa oral.

3.3 Recursividad: definición y estructura

La recursividad es una técnica mediante la cual un método o función se llama a sí mismo para resolver una versión más pequeña del mismo problema. Roberts (2006) plantea la recursividad como una forma de pensamiento: en lugar de resolver todo de una sola vez, se busca una versión más simple y se confía en que la misma regla resuelva el resto. Esta idea es común en matemáticas, por ejemplo en el factorial de un número, donde $n!$ se define como n multiplicado por $(n - 1)!$, hasta llegar al caso base $0! = 1$.

Todo algoritmo recursivo necesita dos partes. La primera es el caso base, que detiene la recursión y entrega una respuesta directa. La segunda es el caso recursivo, que reduce el problema y vuelve a llamar al mismo método. Si falta el caso base, la función se llama indefinidamente y puede provocar un error de desbordamiento de pila. Si el caso recursivo no reduce el problema, aunque exista caso base, es posible que nunca se llegue a él. Por eso la recursividad exige disciplina: cada llamada debe acercarse de forma clara a la condición de parada.

En el proyecto, la recursividad no se usa para avanzar generaciones completas, sino para contar los 8 vecinos de una celda. Normalmente, ese recorrido se resolvería con dos ciclos anidados que revisan las posiciones alrededor de la celda. Sin embargo, el requisito técnico indica que no se permite usar un ciclo `for` o `while` para recorrer las 8 posiciones vecinas. La alternativa es guardar las 8 direcciones en una estructura y recorrerlas con un índice que aumenta en cada llamada recursiva. Cuando el índice llega a 8, significa que ya se revisaron todas las posiciones.

Esta aplicación es adecuada para aprender recursividad porque el problema tiene un tamaño pequeño y controlado. No se trata de una recursión profunda con miles de llamadas, sino de una recursión limitada a 8 pasos por celda. Eso permite que el grupo pueda explicar con claridad el caso base, el caso recursivo y el valor retornado. También evita que la recursividad se use de manera artificial: la llamada recursiva cumple una función real dentro del algoritmo principal, ya que reemplaza el recorrido tradicional de vecinos.

3.4 Caso base, caso recursivo y pila de llamadas

El caso base es la condición que permite terminar una cadena de llamadas recursivas. En el conteo de vecinos, el caso base se alcanza cuando el índice que representa la dirección actual llega al total de direcciones disponibles. Como el Juego de la Vida considera ocho vecinos, el arreglo de direcciones tiene ocho posiciones. Al llegar al índice 8, el método retorna 0 porque ya no queda ninguna posición por revisar. Esta respuesta se va sumando hacia atrás con los valores encontrados en las llamadas anteriores.

El caso recursivo, en cambio, ocurre mientras el índice todavía representa una dirección válida. En ese punto, el algoritmo calcula la nueva fila y la nueva columna a partir de la posición original y de la dirección actual. Luego verifica si esa posición está dentro de la matriz y si contiene un microorganismo vivo. Si ambas condiciones se cumplen, suma 1; si no, suma 0. Después llama al mismo método con el índice aumentado en una unidad. Así, cada llamada resuelve una sola dirección y delega el resto a la siguiente llamada.

Internamente, los lenguajes de programación manejan las llamadas recursivas mediante una pila de llamadas. Cada llamada conserva sus propios valores de parámetros y variables locales hasta que la llamada siguiente termina. En un ejemplo simple, la llamada con índice 0 espera el resultado de la llamada con índice 1; la llamada con índice 1 espera la de índice 2; y así sucesivamente hasta llegar al índice 8. Cuando el caso base retorna, las llamadas pendientes se van resolviendo en orden inverso. Comprender este proceso es importante para explicar por qué la suma final representa el total de vecinos vivos.

En la defensa oral, una pregunta probable es por qué no se cuenta la celda central. La respuesta está en el diseño del arreglo de direcciones. Las direcciones vecinas incluyen combinaciones como $(-1, -1)$, $(-1, 0)$, $(0, 1)$ y $(1, 1)$, pero no incluyen $(0, 0)$. Si se incluyera $(0, 0)$, el algoritmo revisaría la misma celda que se está evaluando y podría contar un microorganismo como vecino de sí mismo. Eso sería incorrecto para las reglas del Juego de la Vida. Por eso la exclusión de la dirección central es una decisión necesaria de diseño.

3.5 Recursividad, iteración y complejidad

La recursividad y la iteración pueden resolver muchos problemas similares, pero no siempre conviene usarlas de la misma manera. La iteración utiliza ciclos como `for` o `while` para repetir instrucciones, mientras que la recursividad repite el proceso mediante llamadas al mismo método. Desde el punto de vista del resultado, ambas técnicas pueden llegar a una solución equivalente. Sin embargo, desde el punto de vista de la explicación y de la estructura del código, la recursividad es especialmente útil cuando el problema se define naturalmente a partir de subproblemas parecidos.

Cormen et al. (2022) explican que muchos algoritmos se analizan mediante recurrencias, especialmente cuando dividen un problema en partes más pequeñas. Aunque el proyecto no usa un algoritmo divide y vencerás, si aplica una idea relacionada: resolver una tarea por medio de pasos sucesivos que se reducen hasta llegar a una condición final. En el conteo de vecinos, el tamaño del problema es el número de direcciones pendientes por revisar. Cada llamada reduce ese número en uno, por lo que el proceso siempre termina si el índice aumenta correctamente.

En términos de complejidad, contar vecinos en el Juego de la Vida tiene costo constante por celda, porque siempre se revisan como máximo ocho posiciones. Si la matriz tiene n por n celdas, el algoritmo completo de una generación debe evaluar n^2 celdas, y para cada una realizar el conteo de vecinos. Como el número de vecinos es fijo, la complejidad por generación se puede describir como $O(n^2)$. Esta explicación es importante porque muestra que usar recursividad en el conteo no cambia el crecimiento principal del algoritmo; simplemente cambia la forma en que se recorre el conjunto de vecinos.

También es importante reconocer una limitación: la recursión usa memoria adicional en la pila de llamadas. En este proyecto la profundidad máxima de la recursión es 8, por lo que el riesgo es mínimo. No obstante, en otros problemas una recursión muy profunda podría consumir

demasiada memoria. Esta observacion demuestra que la recursividad no debe aplicarse sin criterio. Debe usarse cuando mejora la claridad del problema o cuando el requisito técnico lo solicita, siempre cuidando que el caso base sea alcanzable y que cada llamada avance hacia el.

3.6 Autómatas celulares

Un autómata celular es un modelo computacional formado por una red de celdas, donde cada celda tiene un estado y evoluciona según reglas que dependen de sus vecinas. Ilachinski (2001) describe los autómatas celulares como sistemas discretos en espacio y tiempo, caracterizados por interacciones locales y evolución sincronizada. Esto quiere decir que el mundo del modelo avanza por pasos, y en cada paso todas las celdas actualizan su estado según las mismas reglas generales. Aunque cada regla sea simple, el resultado global puede ser difícil de predecir.

La importancia de los autómatas celulares esta en que permiten estudiar fenómenos complejos sin necesidad de construir modelos continuos muy complicados. Wolfram (1983) los utilizo como modelos matematicos simples para investigar autoorganizacion y comportamientos emergentes. En un autómata celular, ninguna celda tiene conocimiento del sistema completo; solo responde a su entorno inmediato. Sin embargo, cuando muchas celdas aplican reglas locales al mismo tiempo, pueden aparecer patrones estables, oscilaciones, estructuras que se desplazan y comportamientos aparentemente inteligentes.

En el Juego de la Vida, la matriz representa el espacio del autómata y cada celda puede estar viva o muerta. La vecindad utilizada es la vecindad de Moore, porque considera las ocho posiciones alrededor de una celda: arriba, abajo, izquierda, derecha y las cuatro diagonales. Esta vecindad es diferente de una vecindad que solo considere cuatro direcciones. El uso de ocho vecinos hace que la evolución sea mas rica, ya que una celda puede verse afectada por microorganismos ubicados diagonalmente.

Para el proyecto programado, la idea de autómata celular ayuda a justificar por que se usa una matriz. Cada posicion de la matriz corresponde a una zona de la reserva natural. El estado de esa zona puede ser vivo o vacio según la presencia de microorganismos. Cada generación representa un instante nuevo de la simulación. De esta forma, el programa combina una estructura de datos sencilla con reglas de actualizacion que producen un comportamiento dinámico.

3.7 Juego de la Vida de Conway

El Juego de la Vida fue presentado al publico por Martin Gardner en Scientific American en 1970, a partir de una propuesta del matematico John Horton Conway. Gardner (1970) lo describio como un juego solitario basado en combinaciones matemáticas, pero con el tiempo se convirtio en un ejemplo clasico de autómata celular. Su atractivo se debe a que no requiere decisiones después del estado inicial: el usuario coloca una configuracion de celdas vivas y, a partir de ese momento, el sistema evoluciona por reglas fijas.

Las reglas clásicas son cuatro. Una celda viva con menos de dos vecinos vivos muere por soledad. Una celda viva con dos o tres vecinos vivos sobrevive. Una celda viva con mas de tres vecinos vivos muere por sobrepoblacion o asfixia. Una celda vacia con exactamente tres vecinos vivos se convierte en una celda viva por nacimiento. Estas reglas son faciles de memorizar, pero generan comportamientos variados. Algunas configuraciones desaparecen, otras se estabilizan, otras oscilan y algunas se desplazan por la matriz.

Adamatzky (2010) presenta el Juego de la Vida como un campo amplio de estudio dentro de los autómatas celulares, especialmente por su relación con patrones, computación no convencional y sistemas discretos. Rendell (2002) incluso muestra la universalidad computacional del Juego de la Vida mediante la construcción de una máquina de Turing dentro de este autómata. Para un proyecto universitario introductorio no es necesario llegar a ese nivel, pero sí es importante comprender que el juego no es solo una curiosidad. Es un modelo que demuestra como reglas simples pueden producir comportamientos complejos.

En el contexto del curso, el Juego de la Vida es útil porque obliga a trabajar con varias habilidades al mismo tiempo. Se usan matrices para almacenar estados, condiciones para aplicar reglas, métodos para separar responsabilidades y recursividad para contar vecinos. Además, el programa debe presentar resultados comprensibles para el usuario. Por eso el juego funciona como puente entre teoría algorítmica y programación práctica.

3.8 Aplicación al ecosistema costarricense

El proyecto solicita que los datos del problema estén ambientados en Costa Rica. Para cumplir este requisito, la simulación puede presentarse como un ecosistema de microorganismos dentro de una reserva natural o área silvestre protegida. El Sistema Nacional de Áreas de Conservación indica que Costa Rica cuenta con un sistema institucional encargado de administrar áreas protegidas terrestres y marinas. Esta referencia permite usar nombres reales como Parque Nacional Corcovado, Parque Nacional Tortuguero, Reserva Biológica Monteverde o Parque Nacional Rincon de la Vieja.

La ambientación costarricense no cambia las reglas matemáticas del Juego de la Vida, pero sí mejora la pertinencia del programa. En lugar de mostrar una matriz abstracta sin contexto, cada celda puede representar una zona de una reserva, un sendero, una laguna, un bosque o un sector específico. Los microorganismos vivos pueden interpretarse como presencia de una colonia microscópica en una zona, mientras que las celdas vacías representan espacios sin actividad en ese momento. Esta narrativa ayuda a que el usuario comprenda el propósito de la simulación.

También es importante aclarar que el programa no pretende modelar biológicamente un ecosistema real con precisión científica. El Juego de la Vida es una simplificación matemática. No considera clima, humedad, alimento, depredadores ni reproducción biológica real. Su valor educativo está en mostrar cómo una población puede cambiar según reglas locales. Por eso, el contexto costarricense debe entenderse como una ambientación académica, no como una predicción ecológica.

Esta distinción puede mencionarse como una limitación del proyecto. La simulación representa un ecosistema de forma simbólica para cumplir el contexto nacional y facilitar la visualización. Aun así, el uso de nombres de áreas protegidas ayuda a que el programa sea menos genérico y demuestra atención a las instrucciones. Además, permite que en la exposición oral se explique por qué se eligió determinada reserva y cómo se relaciona con la matriz.

3.9 Diseño del programa propuesto

El programa propuesto se desarrollará en Java, como lo exige el documento del proyecto. Java es adecuado para este caso porque permite trabajar con arreglos bidimensionales, métodos, clases e interfaces gráficas de escritorio. Para la interfaz se puede usar Swing, una biblioteca oficial de Java para construir ventanas y componentes gráficos. Oracle documenta Swing como una tecnología para crear interfaces gráficas con componentes como ventanas, botones, paneles y

áreas de texto. El uso de Swing no resuelve la lógica del algoritmo, sino que solo facilita la presentación visual.

La estructura básica del programa puede dividirse en cuatro tareas. La primera es crear la matriz con un tamaño válido entre 4x4 y 8x8. La segunda es permitir que el usuario coloque los microorganismos iniciales. La tercera es calcular cada generación aplicando las reglas del Juego de la Vida. La cuarta es mostrar visualmente la matriz y el resumen de nacimientos, sobrevivientes y muertes. Separar estas tareas ayuda a mantener el código ordenado y a explicar cada parte durante la defensa.

La matriz principal puede representarse con un arreglo booleano. El valor verdadero indica que hay un microorganismo vivo y el valor falso indica que la celda está vacía. Para mostrar la diferencia entre un microorganismo que sobrevivió y uno que nació, puede usarse una segunda matriz o una variable visual auxiliar. Esta separación es importante porque el estado matemático solo necesita saber si la celda está viva, pero la interfaz debe comunicar más información al usuario.

Otro punto de diseño es que la siguiente generación debe calcularse a partir de la matriz anterior, no sobre la misma matriz mientras se recorre. Si se cambia una celda inmediatamente, ese cambio podría afectar el conteo de vecinos de otra celda que todavía no ha sido evaluada. Por eso se recomienda crear una nueva matriz para guardar el resultado de la generación. Al final del proceso, la matriz actual se reemplaza por la nueva. Esta técnica respeta la naturaleza sincronizada de los autómatas celulares.

3.10 Método recursivo para contar vecinos

El método recursivo para contar vecinos es el centro técnico del proyecto. Su objetivo es recibir la matriz, una fila, una columna y un índice. La fila y la columna indican la celda que se está evaluando. El índice indica cuál de las ocho direcciones vecinas se está revisando en ese momento. En lugar de usar un ciclo, el método revisa una dirección y luego se llama a sí mismo con el índice aumentado. Así se recorren las ocho direcciones mediante recursión.

La lógica puede explicarse paso a paso. Primero, el método pregunta si el índice ya llegó al tamaño del arreglo de direcciones. Si esto ocurre, retorna 0 porque no quedan vecinos por revisar. Segundo, si todavía hay una dirección válida, calcula la posición vecina sumando el desplazamiento correspondiente a la fila y columna originales. Tercero, verifica si esa posición está dentro de los límites de la matriz. Cuarto, si está dentro y contiene un microorganismo vivo, asigna 1 al vecino actual; en caso contrario asigna 0. Por último, retorna ese valor más el resultado de llamar al método con el siguiente índice.

Esta estructura permite defender claramente la recursión. El caso base es el índice igual a 8. El caso recursivo es la llamada con índice + 1. La reducción del problema consiste en pasar de ocho posiciones pendientes a siete, luego a seis y así sucesivamente. El retorno acumulado produce el total de vecinos vivos. El método no cuenta la celda central porque las direcciones almacenadas no incluyen el desplazamiento (0, 0). De esta forma se evita el error de contar la propia celda como vecina.

La siguiente tabla resume la diferencia entre una solución no aceptada y la solución esperada por el proyecto:

Version no aceptada	Version aceptada
---------------------	------------------

Usar ciclos for o while para recorrer las ocho posiciones vecinas.

El recorrido depende de repetición iterativa.

No cumple el requisito técnico del tema 4.

Usar un método que revise una dirección y luego se llame a sí mismo con índice + 1.

El recorrido depende del caso base y del caso recursivo.

Cumple el requisito porque las ocho direcciones se recorren recursivamente.

3.11 Interpretación de resultados por generación

En cada generación el programa debe mostrar más que una matriz nueva. También debe indicar que ocurrió con los microorganismos. Para cumplir este requisito, el resultado se puede dividir en sobrevivientes, nacimientos y muertes. Un sobreviviente es una celda que estaba viva y continúa viva por tener dos o tres vecinos. Un nacimiento es una celda que estaba vacía y pasa a estar viva porque tiene exactamente tres vecinos vivos. Una muerte es una celda que estaba viva y pasa a estar vacía por soledad o por asfixia.

Esta clasificación mejora la comprensión del usuario porque no solo muestra el resultado final, sino la razón del cambio. Por ejemplo, si en una generación mueren cuatro microorganismos, el usuario puede saber si murieron por tener pocos vecinos o demasiados. Esa información también sirve para la exposición oral, porque permite explicar la relación entre reglas y matriz. En lugar de decir solamente que el programa funciona, el grupo puede mostrar cómo cada regla produce un conteo concreto.

El resumen de generación también refuerza el carácter algorítmico del proyecto. Cada celda se evalúa con el mismo procedimiento, pero el resultado depende del estado local de sus vecinos. Si se registran las posiciones de nacimientos, sobrevivientes y muertes, el programa puede presentar evidencia clara de su cálculo. Esto es útil para probar el ejemplo dado en el PDF y para demostrar que la implementación no se limita a mostrar colores, sino que procesa información.

Para mantener el programa entendible, el resumen puede mostrarse en texto junto a la matriz visual. La matriz permite ver la distribución espacial y el texto permite revisar cantidades y posiciones. Ambas representaciones se complementan. En una defensa oral, esta combinación facilita responder preguntas porque el expositor puede señalar una celda específica y explicar cuántos vecinos tiene, qué regla aplica y por qué cambio o se mantuvo igual.

3.12 Aporte propio del grupo

El aporte propio del grupo consiste en adaptar un modelo matemático clásico a un contexto costarricense y a una restricción técnica específica. El Juego de la Vida existe desde 1970, pero el proyecto no se limita a repetirlo de manera genérica. La propuesta lo presenta como una simulación de microorganismos en una reserva natural, usa nombres asociados a áreas protegidas del país y convierte el conteo de vecinos en un ejercicio de recursividad. Esa adaptación permite demostrar comprensión del tema y no solo reproducción de una idea conocida.

Otro aporte está en la forma de explicar el código. El método recursivo no debe verse como una línea aislada, sino como una decisión de diseño. Se propone separar las direcciones vecinas en un arreglo, excluir la dirección central y recorrer ese arreglo mediante un índice. Esta solución es sencilla, defendible y cumple la restricción de no usar ciclos para recorrer las ocho posiciones vecinas. Además, permite que cualquier integrante explique el caso base y el caso recursivo con un ejemplo concreto.

También se considera aporte propio mostrar los resultados de cada generación con conteos y detalles. En algunos programas del Juego de la Vida solo se observa una animación de celdas, pero el proyecto exige información académica sobre lo que ocurre. Por eso la propuesta incluye nacimientos, sobrevivientes, muertes y total vivo al final. Esta salida ayuda a conectar la teoría con el funcionamiento real del programa.

Finalmente, el grupo debe asumir que el objetivo no es entregar un programa difícil de leer, sino uno que se pueda comprender y defender. En un contexto universitario, un código claro tiene más valor que una solución demasiado compleja. La recursividad elegida debe ser visible, el flujo debe ser ordenado y los nombres de variables deben ayudar a entender el problema. Esa claridad es parte del aporte propio, porque demuestra que la investigación se transformó en una solución programable.

3.13 Relación entre investigación y proyecto programado

La investigación fundamenta directamente la segunda parte del proyecto. El concepto de algoritmo permite definir entradas, procesos y salidas. La recursividad permite cumplir el requisito técnico del conteo de vecinos. Los autómatas celulares explican por qué una matriz puede representar un sistema que evoluciona. El Juego de la Vida aporta las reglas de nacimiento, sobrevivencia y muerte. El contexto costarricense justifica la ambientación de reservas y zonas protegidas.

Esta relación evita que el documento escrito sea una parte separada del programa. Por el contrario, cada sección teórica tiene una aplicación concreta. Cuando se habla de caso base, se piensa en el índice 8 del método recursivo. Cuando se habla de vecindad de Moore, se piensa en las ocho direcciones alrededor de una celda. Cuando se habla de evolución sincronizada, se piensa en calcular una matriz nueva antes de reemplazar la anterior. Esta conexión hace que la exposición sea más coherente.

Para la defensa oral, se recomienda que los integrantes puedan explicar el flujo completo: el usuario elige tamaño, coloca microorganismos, indica generaciones, el programa evalúa celda por celda, cuenta vecinos con recursión, aplica reglas, genera una nueva matriz y muestra resumen. Si se domina esa secuencia, también se pueden responder preguntas más específicas. Por ejemplo, se puede explicar por qué una celda muere por asfixia o por qué no se debe modificar la matriz original durante el recorrido.

En conclusión, la investigación no solo describe conceptos. Sirve como guía para construir y defender el programa. Un buen documento debe demostrar que el grupo entiende el tema, que eligió fuentes confiables y que puede relacionar la teoría con una solución propia en Java. Esa es la función principal de esta primera parte.

3.14 Ejemplo de análisis de una generación

Para comprender el funcionamiento del Juego de la Vida, conviene analizar una generación como si se estuviera haciendo manualmente. Supongase una matriz de 4x4 donde varias celdas están vivas al inicio. El algoritmo no decide el futuro de una celda por intuición, sino por conteo. Para cada posición, primero calcula cuántos vecinos vivos existen alrededor. Después aplica una regla. Si la celda estaba viva y tiene dos o tres vecinos, sobrevive. Si estaba viva y tiene menos de dos, muere por soledad. Si estaba viva y tiene más de tres, muere por asfixia. Si estaba vacía y tiene exactamente tres vecinos, nace un microorganismo.

Este análisis muestra que el proceso es determinístico. Con la misma matriz inicial, siempre se obtiene la misma generación siguiente. No interviene azar ni decisión del usuario después de haber colocado los microorganismos iniciales. Esta característica es importante porque permite probar el programa. Si el grupo usa el ejemplo del enunciado y obtiene los mismos conteos de sobrevivientes, nacimientos y muertes, puede demostrar que la lógica está bien implementada. Si el resultado no coincide, se debe revisar el conteo de vecinos o la aplicación de reglas.

Una celda de esquina tiene menos posiciones válidas alrededor que una celda central, pero el método recursivo puede manejar ambos casos. Cuando una dirección vecina cae fuera de la matriz, simplemente no se cuenta. Por ejemplo, desde la posición (0,0), direcciones como (-1,-1), (-1,0) o (0,-1) están fuera de los límites. El método debe verificar límites antes de consultar la matriz. Esta validación evita errores de índice y permite que el mismo algoritmo funcione en esquinas, bordes y zonas internas.

También es importante distinguir entre el análisis de una celda y el análisis de una generación completa. Una celda solo requiere contar hasta ocho vecinos. Una generación completa requiere repetir ese análisis para todas las celdas de la matriz. Aunque el requisito recursivo se aplica al conteo de vecinos, el recorrido general de filas y columnas puede hacerse de manera normal, porque el enunciado prohíbe específicamente usar ciclos para recorrer las ocho posiciones vecinas. Esta diferencia debe quedar clara en la defensa, ya que no toda repetición del programa tiene que ser recursiva.

3.15 Pruebas y validación del programa

La validación del programa debe realizarse con casos pequeños antes de probar matrices más grandes. Un primer caso puede ser una matriz vacía. Si no hay microorganismos vivos, no debe aparecer ningún nacimiento porque ninguna celda vacía tiene tres vecinos vivos. Un segundo caso puede ser una sola celda viva. En ese escenario, el microorganismo debe morir por soledad en la siguiente generación. Estos casos son simples, pero ayudan a confirmar que las reglas básicas funcionan y que el programa no crea vida sin condiciones válidas.

Otro caso importante es el ejemplo proporcionado en el documento del proyecto. Como el enunciado muestra posiciones iniciales, resultado de generación 1 y resultado de generación 2, ese ejemplo sirve como prueba de referencia. El grupo puede colocar las mismas posiciones y comparar los conteos. Si el programa indica seis sobrevivientes, dos nacimientos y dos muertes en la primera generación, se obtiene una evidencia fuerte de que el algoritmo sigue las reglas esperadas. Si además la segunda generación coincide en cantidad de vivos, la prueba resulta aún más convincente.

Las pruebas también deben revisar límites de entrada. El usuario no debe poder crear una matriz menor que 4x4 ni mayor que 8x8, porque ese es el rango solicitado. Tampoco debe poder colocar microorganismos en posiciones inexistentes. Si la interfaz usa botones, esta validación se simplifica porque el usuario solo puede hacer clic en celdas visibles. Si la interfaz usa campos de texto, el programa debe validar que las filas y columnas ingresadas estén dentro del rango permitido.

Una validación adicional consiste en revisar que el programa no modifique la matriz original mientras calcula una generación. Si lo hiciera, una celda evaluada al inicio podría afectar incorrectamente a una celda evaluada después. La forma correcta es guardar los resultados en una matriz nueva. Esta prueba se puede explicar con un ejemplo: si una celda nace al comienzo del

recorrido, no debe contarse como vecina viva para otra celda dentro de la misma generación, porque su nacimiento pertenece al estado siguiente, no al estado actual.

3.16 Limitaciones y alcance de la simulación

Aunque el Juego de la Vida se presenta con términos biológicos como nacimiento, muerte y sobrevivencia, no debe confundirse con un modelo biológico completo. El programa no calcula nutrientes, clima, reproducción real, competencia entre especies ni condiciones ambientales de una reserva costarricense. Usa reglas matemáticas simples para representar cambios de estado en una matriz. Esta limitación no reduce el valor académico del proyecto, pero debe mencionarse para evitar interpretaciones incorrectas.

Otra limitación es el tamaño de la matriz. El enunciado establece un rango de 4x4 a 8x8, lo cual es adecuado para fines educativos y para una interfaz fácil de observar. Sin embargo, el Juego de la Vida puede estudiarse en matrices mucho más grandes. En matrices amplias aparecen patrones más complejos, como osciladores y estructuras móviles. En este proyecto, el tamaño reducido permite concentrarse en la recursividad y en la explicación de reglas, no en simulaciones extensas.

También existe una limitación relacionada con la recursividad. El método recursivo utilizado cuenta ocho vecinos y por eso tiene profundidad máxima de ocho llamadas. Esto es seguro y fácil de defender. No obstante, no significa que todo el programa deba escribirse recursivamente. Forzar recursión en partes donde no aporta claridad podría hacer el código más difícil de entender. La decisión correcta es aplicar recursión exactamente donde el requisito lo solicita y mantener el resto del programa ordenado con estructuras simples.

Finalmente, el proyecto tiene un alcance educativo. Su finalidad es demostrar comprensión de algoritmos recursivos, matrices, reglas condicionales y simulación por generaciones. No busca reemplazar software científico ni realizar predicciones ambientales reales. Por esa razón, el éxito del proyecto debe evaluarse por el cumplimiento de los requisitos, la claridad del código, la exactitud de las reglas y la capacidad de los integrantes para explicar el funcionamiento sin depender de memorizar líneas aisladas.

3.17 Preparación para la defensa oral

La defensa oral debe enfocarse en explicar el razonamiento del algoritmo. Una buena preparación consiste en que cada integrante pueda responder tres preguntas esenciales. La primera es donde está el caso base. La segunda es que ocurre cuando el índice llega a 8. La tercera es por qué no se cuenta la celda central. Estas preguntas aparecen directamente relacionadas con el requisito técnico del tema, por lo que deben responderse con seguridad y con referencia al código.

También conviene preparar una explicación sencilla de la matriz. La matriz representa la reserva o ecosistema dividido en zonas. Cada posición tiene coordenadas de fila y columna. Cada celda puede estar viva o vacía. Al avanzar una generación, el programa revisa todas las posiciones y crea una nueva matriz de resultados. Esta explicación ayuda a que el público entienda el programa antes de entrar en detalles de recursividad.

Para explicar el método recursivo, se puede usar una analogía con una lista de ocho direcciones. El método revisa la primera dirección y luego le pide al mismo método que revise la siguiente. Cuando ya no quedan direcciones, retorna cero. Después, cada llamada suma su resultado parcial.

Esta forma de explicarlo evita tecnicismos innecesarios y muestra que el grupo entiende el flujo. Si el profesor pide mayor detalle, se puede mencionar la pila de llamadas y el retorno acumulado.

Por ultimo, la exposicion debe conectar investigacion y programa. No basta con decir que el Juego de la Vida usa reglas. Se debe mostrar como esas reglas se transforman en condiciones dentro del código. No basta con decir que hay recursividad. Se debe senalar el método específico, el parámetro índice, el caso base y la llamada con índice + 1. Esta relación entre teoria e implementación es lo que demuestra aprendizaje real y reduce el riesgo de que el trabajo parezca copiado o no comprendido.

4. Conclusiones

La recursividad es una técnica fundamental para resolver problemas que pueden descomponerse en versiones mas pequenas de si mismos. Su correcta aplicación depende de reconocer el caso base, el caso recursivo y la forma en que cada llamada se aproxima a la condición de parada. En el proyecto del Juego de la Vida, esta técnica se aplica de manera concreta al conteo de los ocho vecinos de cada celda. La recursión permite cumplir el requisito técnico del tema 4 y, al mismo tiempo, ofrece una oportunidad para explicar con claridad como se acumula un resultado a traves de llamadas sucesivas.

El Juego de la Vida de Conway demuestra que reglas simples pueden generar comportamientos complejos cuando se aplican de forma sincronizada sobre una matriz. Cada celda toma decisiones locales según sus vecinos, pero el conjunto produce una evolución global de nacimientos, sobrevivencias y muertes. Esta característica convierte al juego en un ejemplo apropiado para estudiar algoritmos, estructuras de datos y simulación. Además, al ambientarlo en una reserva natural costarricense, el proyecto adquiere un contexto propio y cumple la indicacion de usar datos y escenarios relacionados con Costa Rica.

La investigacion también permite concluir que un buen programa no depende solamente de escribir código, sino de comprender el problema antes de implementarlo. Para defender el proyecto, el grupo debe poder explicar por que se usa una matriz, por que se crea una nueva generación sin modificar inmediatamente la anterior, por que el conteo de vecinos es recursivo y por que la celda central no se considera vecina de si misma. Si estos puntos se dominan, la implementación en Java se vuelve mas clara, mas ordenada y mas fácil de justificar ante preguntas del profesor o de los companeros.

5. Referencias bibliográficas

- Adamatzky, A. (Ed.). (2010). *Game of Life Cellular Automata*. Springer.
<https://doi.org/10.1007/978-1-84996-217-9>
- Adamatzky, A. (Ed.). (2018). *Cellular Automata: A Volume in the Encyclopedia of Complexity and Systems Science, Second Edition*. Springer. <https://doi.org/10.1007/978-1-4939-8700-9>
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). The MIT Press. <https://mitpress.mit.edu/9780262046305/introduction-to-algorithms/>
- Gardner, M. (1970). Mathematical Games: The fantastic combinations of John Conway's new solitaire game Life. *Scientific American*, 223(4), 120-123.
<https://doi.org/10.1038/scientificamerican1070-120>
- Ilachinski, A. (2001). *Cellular Automata: A Discrete Universe*. World Scientific.
https://books.google.com/books/about/Cellular_Automata_A_Discrete_Universe.html?id=BPY7DQAAQBAJ
- Oracle. (s. f.). *Trail: Creating a GUI With Swing*. The Java Tutorials.
<https://docs.oracle.com/javase/tutorial/uiswing/index.html>
- Rendell, P. (2002). Turing Universality of the Game of Life. En A. Adamatzky (Ed.), *Collision-Based Computing* (pp. 513-539). Springer. https://doi.org/10.1007/978-1-4471-0129-1_18
- Roberts, E. (2006). *Thinking Recursively with Java*. John Wiley & Sons.
<https://cs.stanford.edu/people/eroberts/books/ThinkingRecursively/index.html>
- Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley.
<https://algs4.cs.princeton.edu/>
- Sistema Nacional de Areas de Conservacion. (s. f.). *Areas Silvestres Protegidas*.
<https://www.sinac.go.cr/ES/asp/Paginas/default.aspx>
- Sistema Nacional de Areas de Conservacion. (s. f.). *Sistema Nacional de Areas de Conservacion*.
<https://www.sinac.go.cr/ES/conozca/Paginas/default.aspx>
- Von Neumann, J., & Burks, A. W. (Ed.). (1966). *Theory of Self-Reproducing Automata*. University of Illinois Press. https://openlibrary.org/works/OL17134241W/Theory_of_self-reproducing_automata
- Wolfram, S. (1983). Statistical mechanics of cellular automata. *Reviews of Modern Physics*, 55(3), 601-644. <https://doi.org/10.1103/RevModPhys.55.601>